

Nombre y Apellido	L.U.	N° Orden	Materia y turno	
			☐ TM	☐ TDA
			☐ TN	☐ Algo3

Duración: 4 horas. Este examen es individual y a libro cerrado. La nota de aprobación es 60/100 puntos.

1. Preguntas de opción multiple

Cada pregunta tiene 4 opciones y una sola opción correcta. Las respuestas correctas (bien) suman 6 puntos. Las respuestas incorrectas (mal) restan 3 puntos. Las preguntas no respondidas no se consideran como incorrectas y suman 0 puntos.

Marcar en el cuadrado la opción que consideren correcta.

1. Considere el Problema del Viajante de Comercio (TSP): dado un grafo completo no dirigido $G = (V, E)$ con n nodos (ciudades), donde cada arista (i, j) tiene asociado un costo no negativo $c(i, j)$ que representa la distancia entre las ciudades i y j , se debe encontrar un ciclo que visite todas las ciudades exactamente una vez y regrese a la ciudad inicial, minimizando la distancia total recorrida. ¿Cuál es la complejidad temporal del algoritmo de fuerza bruta (asumiendo que la única poda es no repetir ciudades)?

☐ a. $\Theta(n)$ ☐ b. $\Theta(n^2)$ ☒ c. $\Theta(n!)$ ☐ d. $\Theta(2^n)$

2. ¿Cuál de los siguientes NO es un componente típico de un algoritmo de backtracking?

☐ a. Una función que verifica si una solución parcial puede conducir a una solución válida.
☐ b. Una función que verifica si hemos alcanzado una solución completa.
☐ c. Un mecanismo para deshacer decisiones previas.
☒ d. Un algoritmo de ordenamiento para optimizar las soluciones.

3. Si ejecutamos DFS en un grafo dirigido acíclico (DAG), ¿qué propiedad tendrá la secuencia de vértices ordenados por tiempo de finalización en orden decreciente?

☐ a. Será un recorrido en anchura del grafo.
☒ b. Será un ordenamiento topológico del grafo.
☐ c. Será un recorrido en orden creciente de distancia al origen.
☐ d. Ninguna de las anteriores.

4. ¿Cuál es la complejidad temporal del algoritmo DFS cuando se implementa utilizando una lista de adyacencia? En este caso, se pide NO asumir que V es $O(E)$.

☐ a. $\Theta(V)$ ☐ b. $\Theta(E)$ ☒ c. $\Theta(V + E)$ ☐ d. $\Theta(V.E)$

2. Preguntas abiertas de respuesta corta

Respuesta Bien: +8 puntos. Respuesta Regular, Mal o sin completar: +0 puntos.

5. En un algoritmo de fuerza bruta que genera y almacena todas las permutaciones posibles de N elementos, ¿cuál es la complejidad temporal y espacial? Justifique brevemente.

Un algoritmo de fuerza bruta que genera todas las permutaciones debe crear exactamente $N!$ permutaciones diferentes. Cada permutación requiere $O(N)$ tiempo para generarse y $O(N)$ espacio para almacenarse. Por tanto, el tiempo total es $O(N!)$ y el espacio para almacenar todas las permutaciones es $O(N!N)$.

6. Dada una secuencia de n matrices $A_1, A_2, \dots, A_n$ que deben multiplicarse en orden, el objetivo es determinar la forma óptima de parentizar la multiplicación (aprovechar la propiedad asociativa) para minimizar el número total de multiplicaciones escalares requeridas. Si lo resolvemos mediante programación dinámica, ¿Qué se calcula en cada estado de la DP (respecto del número de multiplicaciones escalares)?

En cada estado $DP[i][j]$ se calcula el número mínimo de multiplicaciones escalares necesarias para multiplicar la secuencia de matrices desde A_i hasta A_j . Específicamente, $DP[i][j]$ representa el costo óptimo (en multiplicaciones escalares) de calcular el producto $A_i \times A_{i+1} \times \dots \times A_j$, considerando todas las posibles formas de parentizar esta multiplicación en cadena.

3. Ejercicios a Desarrollar

Bien: +15 pts. Ejercicio Mal o sin completar: +0 puntos.

7. Dado un grafo dirigido acíclico (DAG) con N nodos y M aristas, donde cada arista (u, v) tiene un peso $w(u, v)$ que puede ser positivo, negativo o cero. Se quiere encontrar el camino de peso máximo desde un nodo origen s hasta cada uno de los demás nodos del grafo.

Diseñe un algoritmo que solucione este problema con una complejidad temporal de $O(N + M)$. Explique detalladamente el algoritmo.

Sugerencia: considere cómo podría utilizarse el ordenamiento topológico del grafo y una adaptación de algún algoritmo de camino más corto.

- a) Ordenamiento Topológico $O(N + M)$
- b) Inicialización $O(N)$ $dist[s] = 0$ (distancia del origen a sí mismo) $dist[v] = -\infty$ para todos los demás nodos (inalcanzables inicialmente)
- c) Relajación de Aristas $O(M)$

Procesamos nodos en orden topológico

Para cada arista (u, v) , si $dist[u] + peso(u, v) > dist[v]$, actualizamos $dist[v]$

Al procesar los nodos en orden topológico, garantizamos que:

- Cuando procesamos un nodo u , ya hemos calculado la distancia máxima desde s hacia u .
- No necesitamos volver a visitar nodos porque no hay ciclos (es un DAG).
- La propiedad de optimalidad se mantiene: si hay un camino de peso máximo de s a v que pasa por u , entonces el subcamino de s a u también debe ser de peso máximo.

Ordenamiento topológico: $O(N + M)$

Calcular grados de entrada: $O(M)$

Procesar nodos: $O(N)$

Procesar aristas: $O(M)$

Inicialización: $O(N)$

Relajación: $O(M)$

Cada arista se examina exactamente una vez

Total: $O(N + M)$

8. Dado un rompecabezas deslizante de 3×3 (8-puzzle), definimos el estado del rompecabezas como una configuración específica de las 8 fichas numeradas y el espacio vacío en la grilla. El rompecabezas se considera resuelto cuando las fichas están ordenadas del 1 al 8 y el espacio vacío está en la esquina inferior derecha. Una jugada válida consiste en deslizar una ficha adyacente al espacio vacío hacia dicho espacio.

Se pide modelar el rompecabezas con un grafo donde:

- a) Cada nodo represente un estado del rompecabezas.
- b) La distancia entre dos nodos u y v sea igual a la cantidad de jugadas necesarias para convertir el estado representado por u en el estado representado por v .

Grafo no dirigido donde cada nodo representa un estado específico del rompecabezas (una configuración de las 8 fichas numeradas y el espacio vacío). Dos nodos están conectados si sus estados difieren en exactamente una jugada válida (mover una ficha adyacente al espacio vacío). La distancia entre nodos u y v es el número mínimo de jugadas para transformar el estado u en el estado v .

9. Sea el problema de las N -Reinas el problema de colocar N reinas en un tablero de ajedrez $N \times N$ tal que ninguna reina ataque a otra. Sean estas dos soluciones con Backtracking a dicho problema:

```

def N_Reinas(estado: list[int], fila: int) -> int:
    if fila == N:
        return int(es_valido(estado))
    retorno = 0
    for col in range(N):
        estado[fila] = col
        retorno += N_Reinas(estado, fila + 1)
    return retorno

def N_Reinas_poda(estado: list[int], fila: int) -> int:
    if fila == N:
        return 1
    retorno = 0
    for col in range(N):
        estado[fila] = col
        if es_valido_parcial(estado, fila):
            retorno += N_Reinas_poda(estado, fila + 1)
    return retorno

```

Demostrar que:

a) La cantidad de estados visitados por N_Reinas es $\Theta(N^N)$.

Análisis del árbol de recursión:

Profundidad: N niveles (una llamada por cada fila).

Factor de ramificación: N en cada nivel (una llamada por cada columna).

Estructura: En cada nodo interno, se generan exactamente N llamadas recursivas.

El número total de nodos en el árbol de recursión es:

Nivel 0: 1 nodo (raíz)

Nivel 1: N nodos

Nivel 2: N^2 nodos

...

Nivel i: N^i nodos

Total de estados = $\sum_{i=0}^{N-1} (N^i)$

Aplicando la fórmula dada: $\sum_{i=0}^N (r^i) = (r^{(n+1)} - 1)/(r - 1) \in \Theta(r^n)$ para $r > 1$ Con $r = N$ y $n = N - 1$:
 $\sum_{i=0}^{N-1} (N^i) = (N^N - 1)/(N - 1) \in \Theta(N^N)$

b) La cantidad de estados visitados por N_Reinas_poda es $O(N!)$.

Análisis con poda:

En cada fila i, el número de posiciones válidas está limitado por las restricciones:

Columnas: No puede estar en la misma columna que reinas anteriores

Diagonales: No puede estar en las mismas diagonales que reinas anteriores

Cota superior del factor de ramificación:

Fila 0: máximo N opciones

Fila 1: máximo N - 1 opciones (al menos 1 columna bloqueada)

Fila 2: máximo N - 2 opciones (al menos 2 posiciones bloqueadas)

...

Fila i: máximo N - i opciones

En el peor caso el número máximo de hojas en el árbol podado es: $N \times (N - 1) \times (N - 2) \times \dots \times 1 = N!$

Puede usar lo siguiente sin necesidad de demostrarlo:

$$\blacksquare \sum_{i=0}^n r^i = \frac{r^{n+1}-1}{r-1} \in \Theta(r^n) \text{ para } r > 1 \quad \blacksquare \sum_{i=0}^n i! \in \Theta(n!)$$

10. Dado un grafo dirigido ponderado $G = (V, E)$ con $|V| = n$ vértices y una secuencia de vértices $v_1, v_2, \dots, v_k$, se quiere encontrar un camino de costo mínimo que visite los vértices de la secuencia en el orden dado, pudiendo pasar por vértices intermedios en el proceso. Se requiere un algoritmo para calcular la longitud de un camino (o ciclo) mínimo que tenga como subsecuencia a esos nodos. Debe tener una complejidad temporal de $O(n^3 + k)$.

El algoritmo resuelve el problema en dos fases principales:

Fase 1: Preprocesamiento con Floyd-Warshall $O(n^3)$

Calcula la distancia mínima entre todos los pares de vértices del grafo

Esto nos permite conocer el costo mínimo de ir de cualquier vértice a cualquier otro, considerando todos los posibles caminos intermedios.

Fase 2: Cálculo del camino siguiendo la secuencia $O(k)$.

Itera sobre la secuencia de vértices $v_1, v_2, \dots, v_k$

Para cada par consecutivo (v_i, v_{i+1}) :

Si $v_i = v_{i+1}$: el costo es 0 (según la regla especificada).

Si $v_i \neq v_{i+1}$: usa la distancia mínima calculada por Floyd-Warshall.

Suma todos estos costos para obtener el costo total.

Floyd-Warshall garantiza que encontramos la distancia más corta entre cualquier par de vértices, considerando todos los caminos posibles con vértices intermedios.

El algoritmo respeta estrictamente el orden dado en la secuencia, calculando el costo de ir de v_i a v_{i+1} en ese orden.

Cuando un vértice se repite consecutivamente, el costo es 0, como se especifica en el problema.

Complejidad: $O(n^3)$ para Floyd-Warshall + $O(k)$ para el cálculo del camino = $O(n^3 + k)$ total.